

Electronic Store Backend

API Testing Guide — Postman Walkthrough

This guide walks through testing the core workflows of the Electronic Store Backend API using Postman — from user registration and JWT authentication, through product and category management, to cart and order operations. Each step shows the HTTP method, endpoint, sample request payload, and expected response.

Base URL (local): <http://localhost:9091> | Live:
<https://electronic-store-backend-pgl0.onrender.com>

Security note: sample values below (emails, passwords, tokens, IDs) are placeholders for demonstration only and do not correspond to live credentials.

1. User Registration & Authentication

Step 1: Register a New User

POST /users

Request Body

```
{
  "name": "Test User",
  "email": "test.user@example.com",
  "password": "*****",
  "gender": "Male",
  "about": "Testing",
  "imageName": "default.png"
}
```

Response

```
{
  "userId": "",
  "name": "Test User",
  "email": "test.user@example.com",
  "password": "",
  "gender": "Male",
  "about": "Testing",
  "imageName": "default.png",
  "provider": "SELF",
  "roles": [
    { "roleId": "", "name": "ROLE_NORMAL" }
  ]
}
```

Step 2: Generate JWT Access Token

POST

/auth/generate-token

Request Body

```
{
  "email": "test.user@example.com",
  "password": "*****"
}
```

Response

```
{
  "token": "",
  "refreshToken": {
    "id": 1,
    "token": "",
    "expiryDate": ""
  },
  "user": { "...": "user details as above" }
}
```

Note: Copy the token value — it is required as a Bearer token for all secured endpoints below.

Step 3: Authorize Requests in Postman

In Postman, open the **Authorization** tab for a request, select type **Bearer Token**, and paste the token generated above. This applies the JWT to the request's Authorization header.

2. Category & Product Management

Step 4: Create a Category

POST

/categories

Request Body

```
{
  "title": "Electronics",
  "description": "Electronic Products",
  "coverImage": "electronics.png"
}
```

Response

```
{
  "categoryId": "",
  "title": "Electronics",
  "description": "Electronic Products",
  "coverImage": "electronics.png"
}
```

Step 5: Create a Product

POST

/products

Request Body

```
{
  "title": "Samsung Galaxy S25",
  "description": "Latest Samsung flagship mobile",
  "price": 80000,
  "discountedPrice": 75000,
  "quantity": 25,
  "live": true,
  "stock": true,
  "productImageName": "default.png"
}
```

Response

```
{
  "productId": "",
  "title": "Samsung Galaxy S25",
  "price": 80000,
  "discountedPrice": 75000,
  "quantity": 25,
  "live": true,
  "stock": true,
  "category": null
}
```

Step 6: Assign Product to Category

PUT

/categories/{categoryId}/products/{productId}

Response

```
{
  "productId": "",
  "title": "Samsung Galaxy S25",
  "category": {
    "categoryId": "",
    "title": "Electronics"
  }
}
```

Step 7: Get Products by Category

GET

/categories/{categoryId}/products

Response

```
{
  "content": [ { "productId": "", "title": "Samsung Galaxy S25", "...": "..." } ],
  "pageNumber": 0,
  "pageSize": 10,
  "totalElements": 1,
  "totalPages": 1,
  "lastPage": true
}
```

Step 8: Upload Product Image

POST

/products/image/{productId}

Request Body

```
form-data
  key: productImage  type: file  value:
```

Response

```
{
  "imageName": ".jpg",
  "message": "Product image is successfully uploaded !!",
  "success": true,
  "status": "CREATED"
}
```

3. Cart & Order Workflow

Step 9: Add Product to Cart

POST

/carts/{userId}

Request Body

```
{
  "productId": "",
  "quantity": 2
}
```

Response

```
{
  "cartId": "",
  "items": [
    { "cartItemId": 1, "quantity": 2, "totalPrice": 150000 }
  ]
}
```

Step 10: Get Cart by User

GET

/carts/{userId}

Response

```
{
  "cartId": "",
  "user": { "...": "..." },
  "items": [ { "cartItemId": 1, "quantity": 2, "totalPrice": 150000 } ]
}
```

Step 11: Create Order

POST

/orders

Request Body

```
{
  "cartId": "",
  "userId": "",
  "billingAddress": "Warangal, Telangana",
  "billingPhone": "9876543210",
  "billingName": "Test User"
}
```

Response

```
{
  "orderId": "",
  "orderStatus": "PENDING",
  "paymentStatus": "NOTPAID",
  "orderAmount": 150000,
  "orderItems": [ { "orderId": 1, "quantity": 2, "totalPrice": 150000 } ]
}
```

Step 12: Get Orders by User

GET

/orders/users/{userId}

Response

```
[
  {
    "orderId": "",
    "orderStatus": "PENDING",
    "paymentStatus": "NOTPAID",
    "orderAmount": 150000
  }
]
```

4. Admin Operations

The endpoints below require an account with **ROLE_ADMIN**. Generate an admin token the same way as Step 2, using admin credentials, then set it as the Bearer token before calling these routes.

Step 13: Get All Orders (Admin)

GET`/orders`

Response

```
{
  "content": [
    { "orderId": "", "orderStatus": "PENDING", "paymentStatus": "NOTPAID", "orderAmount": 150000 }
  ],
  "pageNumber": 0,
  "pageSize": 10,
  "totalElements": 1,
  "totalPages": 1,
  "lastPage": true
}
```

Step 14: Update Order & Payment Status (Admin only)

PUT`/orders/{orderId}`

Request Body

```
{
  "orderStatus": "APPROVED",
  "paymentStatus": "PAID"
}
```

Response

```
{
  "orderId": "",
  "orderStatus": "APPROVED",
  "paymentStatus": "PAID",
  "orderAmount": 150000
}
```

Interactive API Documentation

Full Swagger/OpenAPI documentation with a live "Try it out" console is available at:

```
https://electronic-store-backend-pgl0.onrender.com/swagger-ui/index.html
```

Note: the app is hosted on a free-tier instance and may take up to a minute to spin up on first request.